# JavaScript and TypeScript compact cheatsheet

## STEP 1 Foundations

### 1.1 Getting Started with JavaScript

**What JavaScript is and where it runs** JavaScript is the programming language of the web. Every modern browser has a JavaScript engine built in, so the same code runs on a phone, a laptop and a smart TV. Outside the browser, Node.js , Deno and Bun run JavaScript on servers, laptops and CI machines.

**Setting up Node.js and an editor** To follow along on your own machine you need two things: Node.js and a code editor. Every example on this page also runs inside your browser with the ✏️ Edit and run button, so you can start right now.

**Your first program** A JavaScript program is a list of statements that run from top to bottom. console.log prints whatever you give it. Several values are printed with a space between them.

**The console toolbox** The console is your first debugging tool and the place where test runners print what they find. It can do far more than log .

**JavaScript in a web page** In a browser, JavaScript can read and change the page. The page is a tree of objects called the DOM (Document Object Model). document.querySelector finds an element with a CSS selector, the same kind of selector a test tool uses to find a button.

**How the engine runs your code** Running a script happens in a few steps, and knowing them explains several surprises you will meet later.

**Reading error messages** Errors are not a failure on your part. They are the fastest way to learn what the program really did. Every error has a name , a message and a stack trace that lists where it happened.

**Real world scenario: summarise a test run** The script below does exactly that with only what this chapter covered: an array of objects, a loop, a template literal and the console.

EXAM A line break right after `return` makes JavaScript insert a semicolon there. The function returns `undefined` and the object below is never reached. Always start the returned value on the same line as `return`.

### 1.2 How a JavaScript Program Runs Inside the Engine

**The journey of a JavaScript program** When you press Run, your .js file is only text. Two programs work together to turn that text into output. The engine understands the JavaScript language.

**Step 1: loading the source text** The engine cannot read files or download anything itself. The runtime fetches the text and hands it over.

**Step 2: the scanner cuts text into tokens** The first thing the engine does with the text is scanning (also called lexing or tokenising). It reads the characters from left to right and groups them into tokens , the smallest pieces that mean something.

**Step 3: the parser builds a syntax tree** The parser reads the tokens and checks them against the grammar of JavaScript. If they fit, it builds an abstract syntax tree (AST): a tree of nodes such as VariableDeclaration , BinaryExpression and CallExpression .

**Step 4: scopes and the creation phase** Before a script or a function runs, the engine sets up an execution context for it. This happens in two phases.

**Step 5: Ignition turns the tree into bytecode** V8 does not run the tree directly. Its interpreter, Ignition , first turns the tree into bytecode : a list of small, simple instructions. Bytecode is much smaller than machine code, so it starts fast and saves memory on phones.

**Step 6: running: the call stack, the heap and garbage collection** While the bytecode runs, the engine uses two areas of memory.

**Step 7: hot code gets compiled: Sparkplug, Maglev and TurboFan** Interpreting bytecode is quick to start but slower to run. So V8 watches which functions run often and compiles them to machine code while the program is running.

**Step 8: the runtime, Web APIs and the event loop** The engine runs one piece of code at a time on one thread. So how does a page wait for a server without freezing? It does not wait.

**Modules: load, link, then evaluate** An ES module is not run the moment it is read. The runtime and the engine go through three phases for the whole graph of files.

**Real world scenario: a frozen page on sale day** The cart runs a long synchronous calculation over every item. While it runs, the call stack is never empty, so the event loop cannot run click handlers, timers or painting. The code below measures this with a timer that should fire after 50 ms.

### 1.3 Variables, Types and Operators

**Variables with let and const** A variable is a name for a value. Modern JavaScript has two keywords for creating one.

**The types of value** Every value in JavaScript is one of eight types. Seven are primitives , simple values that cannot be changed. Everything else is an object .

**Numbers, NaN and BigInt** JavaScript has one number type for whole numbers and decimals. It stores them in binary, so some decimals cannot be stored exactly.

**Type conversion and coercion** Conversion is when you change a type on purpose. Coercion is when JavaScript does it for you behind the scenes. Both follow fixed rules.

**Equality: ==, === and Object.is** JavaScript has three ways to ask "are these the same".

**Operators** Operators combine values. Most of them you know from maths. A few are special to JavaScript and very handy.

**Real world scenario: clean up data from an API** Every tool from this chapter is here: ?? for missing values, ?. for the optional discount, explicit Number conversion, and a check with Number.isNaN for anything that went wrong.

EXAM `const` does not freeze an object. It only stops the name from being reassigned. Use `Object.freeze(obj)` when the contents must not change either.

TRAP `typeof null` is `"object"` and `typeof []` is `"object"` too. Check for null with `value === null` and for arrays with `Array.isArray(value)`.

EXAM Values read from an input field, a URL or an environment variable are always strings. `process.env.HEADLESS` set to `"false"` is truthy. Compare it with the text: `process.env.HEADLESS === "true"`.

### 1.4 Control Flow, Loops and Errors

**Making decisions with if and else** An if statement runs a block only when a condition is truthy. else if adds more conditions, checked from the top, and the first one that matches wins.

**switch and lookup tables** switch compares one value against many cases with === . Cases without a break or return fall through into the next one, which is useful for shared results and a classic bug otherwise.

**Loops** Loops repeat work. Pick the loop by what you are repeating over.

**Loops with async steps** Test code is full of steps that take time: open a page, click, wait for a response. These steps return promises and are written with await (chapter 11 explains how they work). Loops and await have one trap that breaks many first test suites.

**Throwing and catching errors** Code signals a problem by throwing an error. The error travels up through the functions that called it until a try block catches it. If nothing catches it, the program stops.

**Polling and retry loops** Two loops sit at the centre of every test framework. You will rarely write them yourself, but knowing how they work explains timeouts, flaky tests and the options every tool offers.

**Real world scenario: data driven password checks** This is data driven testing : the logic of the check is written once and the cases live in a table. Adding a case means adding one row.

TRAP A `while` loop whose condition never becomes false runs forever and freezes the page or the test. Every polling loop needs a timeout, as shown later in this chapter.

### 1.5 Functions, Scope and Closures

**Declaring functions** A function is a named, reusable piece of code. It takes parameters , does some work and returns a value. A function without a return returns undefined .

**Function expressions and arrow functions** A function can also be written as an expression and stored in a variable. The shorter arrow function syntax is the most common form in modern code and in test files.

**Scope, hoisting and the temporal dead zone** Scope decides where a name can be seen. JavaScript looks for a name in the current block, then in each enclosing block and function, out to the global scope. This chain is fixed by where the code is written , which is called lexical scope.

**Closures** A closure is a function together with the variables it could see when it was created. Those variables stay alive as long as the function does, even after the outer function has returned. Closures give you private state without classes.

**this, call, apply and bind** this is decided by how a function is called, not where it is written. Arrow functions are the exception: they never get their own this and use the one from the surrounding code.

**Higher order functions** A higher order function takes a function as an argument or returns one. You already used one: array.map(fn) . They let you write the shape of a task once and plug in the details.

**Recursion and the call stack** A recursive function calls itself on a smaller piece of the problem. It needs a base case that stops the calls. Every call waits on the call stack until the one it made returns.

**Real world scenario: a test data factory**

EXAM With `var`, every function created in the loop shares one variable, which has its final value by the time they run. `let` creates a new binding for each turn. This was a famous interview question before `let` existed.

## STEP 2 Working with data

### 2.1 Objects and Arrays

**Objects** An object groups related values under names called keys or properties . Almost everything you test is an object: a user, an API response, a configuration, a page element.

**Destructuring** Destructuring pulls values out of objects and arrays into variables in one line. It can rename, give defaults, skip items and collect the rest.

**Spread, copies and immutability** The spread syntax ... copies the items of an array or the keys of an object into a new one. When keys repeat, the later one wins, which makes it perfect for "defaults plus overrides".

**Arrays** An array is an ordered list. Indexes start at 0, length is the number of items and at(-1) reads from the end.

**Array methods: map, filter, reduce and friends** These methods replace most hand written loops. Each one takes a function and calls it for every item.

**Grouping, counting and unique values** Reports need data grouped and counted. ES2024 added Object.groupBy and Map.groupBy . reduce still handles any custom shape, and a Set removes duplicates.

**Real world scenario: verify a sorted product listing** The check found two real problems in this data: the second USB Hub at ₹1,299 comes after Keyboard at ₹2,499, so the order breaks at position 3, and USB Hub is listed twice.

EXAM Spread makes a SHALLOW copy: the outer object is new, but nested objects and arrays are shared. Use `structuredClone` for a full deep copy.

TRAP `sort()` without a compare function sorts as TEXT , so `100` comes before 9. For numbers always pass `(a, b) => a - b`. It also changes the original array. ES2023 added `toSorted`, `toReversed`, `toSpliced` and `with`, which leave the original alone.

### 2.2 Strings, Regex, Numbers, Dates and JSON

**Working with strings** A string is text. Strings are immutable : every method returns a new string and the original never changes. Indexes start at 0.

**Regular expressions** A regular expression describes a pattern of text. It is written between slashes, followed by optional flags .

**Math and number formatting** The Math object holds rounding, min and max, powers and random numbers. Intl.NumberFormat turns numbers into the exact text a user in each country expects.

**Intl: one app, every language and country** A shopping app used in Mumbai, Chicago and Berlin must show the same price, date and sentence in three different ways. Writing those rules by hand is how bugs like "1 items" or "$12,34,567" reach production.

**Dates, time zones and Temporal** The Date object stores one moment as milliseconds since 1 January 1970 UTC. It has well known traps, which is why the new Temporal API was designed. It reached stage 4 in 2026 and is listed for the ES2027 edition.

**JSON** JSON (JavaScript Object Notation) is the text format of almost every API. JSON.stringify turns a value into JSON text and JSON.parse turns text back into a value.

**Real world scenario: validate an order API response**

TRAP `length` counts UTF-16 code units, not visible characters. Emoji and some accents count as 2 or more. Use `[...str].length` for characters, `normalize("NFC")` before comparing and `localeCompare` for human sorting.

TRAP `Date` months start at 0 (January is 0, December is 11), invalid dates such as 30 February roll over silently, and methods without "UTC" use the computer's time zone, so the same test can pass in Bengaluru and fail on a CI server in Frankfurt.

### 2.3 Map, Set, Iterators and Generators

**Map** A Map stores key and value pairs, like an object, with three differences that matter: keys can be any value (objects, numbers, functions), it remembers insertion order, and it has a size .

**Set and set operations** A Set holds unique values. Adding a value twice keeps one copy. ES2025 added the operations from school maths, so comparing two lists no longer needs nested loops.

**WeakMap, WeakSet and WeakRef** Weak collections hold objects without keeping them alive. When nothing else uses the object, the garbage collector may remove it and its entry disappears. They cannot be listed or counted, because their contents can change at any moment.

**Iterators and iterator helpers** An iterable is anything with a Symbol.iterator method: arrays, strings, Maps, Sets, NodeLists and arguments. That method returns an iterator , an object whose next() hands out { value, done } one at a time. for...of , spread and destructuring all use it.

**Generators and async generators** A generator function ( function* ) can pause at every yield and continue later from the same place. Calling it runs nothing. It returns an iterator, and each next() runs the body up to the next yield .

**Binary data: ArrayBuffer, typed arrays and DataView** Most JavaScript works with text and objects, but files, images, network packets, audio and cryptography work with bytes . An ArrayBuffer is a block of raw memory with a fixed size. You never read it directly.

**Symbols** A symbol is a unique value used as a hidden key. Two symbols with the same description are still different, unless they come from the shared registry with Symbol.for . Symbol keys are skipped by Object.keys and JSON.stringify .

**Real world scenario: compare a menu and walk every page** Set operations produce the missing and unexpected lists in one line each, and duplicates are easy to spot once you compare each item's first position with its own. The async generator hides the paging, so the count is a plain loop.

## STEP 3 Structure and tooling

### 3.1 Prototypes, Classes and the Page Object Model

**Objects and prototypes** Every object in JavaScript has a hidden link to another object called its prototype . When you read a property the object does not have, JavaScript follows that link and looks there, then on that object's prototype, and so on.

**Classes** A class is a template for objects that share the same shape and methods. It has a constructor that sets up each new object, methods shared through the prototype, fields that every object gets, getters and setters that look like properties, and static members ...

**Private fields and encapsulation** A field or method whose name starts with # is private . Only code inside the class body can read or change it. It does not show up in Object.keys , in JSON or in the console of other code, and any attempt to read it from outside is a syntax error.

**Inheritance with extends and super** class Dog extends Animal makes Animal.prototype the prototype of Dog.prototype . A Dog gets every Animal method and can override any of them. Inside the subclass, super(...) calls the parent constructor, and super.method() calls the parent version of a method.

**Composition over inheritance** Deep class trees get hard to change: a fix in a parent class affects every child. Many teams prefer composition , where an object is built from small parts it holds or mixes in.

**The Page Object Model** The Page Object Model (POM) is the most common design pattern in UI automation. Each page or component of the app gets a class. The class holds the locators and offers methods with business names such as loginAs or addToCart .

**Real world scenario: test data and an API client** A builder class starts from sensible defaults, and each test changes only what matters to it. Every built user gets a fresh email, so tests never clash.

TRAP In a subclass constructor, `super()` must run before the first use of `this`. Forgetting it throws a `ReferenceError` as soon as the class is used.

### 3.2 Modules, npm, TypeScript and Tooling

**ES modules: import and export** A module is a file with its own scope. Nothing inside it is visible to other files unless it is exported , and nothing from outside is available unless it is imported .

**CommonJS and how the two mix** Before ES modules, Node.js used CommonJS : require() to load and module.exports to share. Older packages, Jest setups, Mocha and many Selenium projects still use it, so you will read both styles.

**npm and package.json** npm installs packages from the npm registry into node_modules and records them in package.json . The lock file ( package-lock.json ) pins the exact version of every package, so every laptop and CI machine installs the same thing.

**TypeScript** TypeScript is JavaScript with types added. You describe the shape of your data once, and the type checker warns you about wrong property names, missing fields and wrong value types while you type, long before a test runs.

**ESLint and Prettier** ESLint reads your code without running it and reports likely bugs and risky habits: unused variables, typos in names, == where === was meant, an async function that never awaits. Rules live in eslint.config.js .

**Environment variables and config** The same tests must run against a laptop, a staging server and sometimes production. The parts that change, such as the base URL, user names and secrets, come from environment variables in process.env .

**Real world scenario: lay out a test project** Source code lives in src . Tests are split by kind under tests . Each kind gets its own npm script, so a developer runs npm run test:unit in seconds while CI runs every script.

TRAP The error `Cannot use import statement outside a module` means a file with `import` is being treated as CommonJS. Add `"type": "module"` to package.json or rename the file to `.mjs`.

TRAP Everything in `process.env` is a string. `"false"` is truthy, so compare with `=== "true"` or `!== "false"`, and turn numbers into numbers. Never commit passwords or tokens: add `.env` to `.gitignore` and use your CI tool's secret store.

## STEP 4 Async and the platform

### 4.1 Asynchronous JavaScript: Promises, async and await

**One thread and the event loop** JavaScript runs your code on one thread : one line at a time. Slow work such as a network request, a timer or reading a file is handed to the browser or Node.js, and your code keeps going. When the slow work finishes, a callback is queued.

**Callbacks** The first way to handle async work was to pass a function to call later. Node.js callbacks receive the error first and the result second.

**Promises** A promise is an object that stands for a result that is not ready yet. It starts pending , then becomes fulfilled with a value or rejected with an error, and never changes again. then runs when it fulfils, catch when it rejects and finally either way.

**async and await** async before a function makes it return a promise. Inside it, await pauses that function until a promise settles and gives you its value, while the rest of the program keeps running. Errors become normal exceptions you catch with try and catch .

**Running many promises: all, allSettled, race, any** Four helpers combine several promises into one. They differ in what counts as done and what happens on a failure.

**Retries, polling and waiting the right way** Tests and scripts often wait for something that happens at an unknown time: an element appears, a job finishes, a service recovers. A fixed sleep guesses the time. It wastes time when the guess is too long and fails when it is too short.

**Real world scenario: a link checker** mapLimit starts 3 workers that take the next URL as soon as they are free. AbortSignal.timeout stops slow requests, and catching each error turns it into a row in the report instead of stopping the run.

TRAP A rejected promise that nobody catches is an UNHANDLED REJECTION . In Node.js it crashes the process, and in test runners it often shows up as a confusing failure in a different test.

### 4.2 The DOM and the Browser

**The DOM tree** When a browser loads HTML, it builds the DOM (Document Object Model): a tree of objects, one for each element, piece of text and comment. JavaScript reads and changes the page only through this tree, and so does every UI automation tool.

**Selectors and locators** A CSS selector describes elements by tag, id, class, attribute and position. The same selector works in document.querySelector , in your stylesheet and in every test tool.

**Changing the page** Elements can be changed in place: their text, classes, attributes and children. textContent sets plain text and is always safe. innerHTML parses HTML, so putting user input into it can run an attacker's code, a bug called cross site scripting (XSS).

**Events** Clicks, key presses, input changes and form submits are events . addEventListener(type, handler) runs your function each time one happens.

**Forms and validation** Form fields hold their current value in value (or checked for checkboxes). HTML attributes such as required , type="email" and min give free validation, which JavaScript reads through checkValidity() and each field's validity .

**Shadow DOM, iframes and observers** Web components hide their inner elements inside a shadow root . A normal querySelector from the document cannot see inside it. An iframe holds a whole separate document, often for payment forms, maps or embedded editors.

**How the browser draws the page** Changing the DOM does not change the screen straight away. The browser collects your changes and then runs its rendering pipeline , usually 60 times a second, between tasks of the event loop (chapter 2).

**Storage and cookies** Browsers keep data for a site in three main places. localStorage keeps strings until they are deleted. sessionStorage keeps them until the tab closes.

**Real world scenario: check an order table** Reading the header cells first turns each row into an object keyed by column name, so the check keeps working if a column is added or moved. The footer does match the rows as shown, so a check of the total alone would have missed this bug.

TRAP Events created by scripts have `isTrusted` set to false, and `element.click()` skips the checks a real user goes through: it works on hidden or covered buttons. That is why Playwright and Selenium move a real mouse and check that the element is visible and not covered before clicking, and why `{ force: true }` clicks can hide real bugs.

TRAP Cookies marked `HttpOnly` cannot be read from JavaScript at all, which protects session tokens from XSS. Test tools read them through the browser instead, for example `context.cookies()` in Playwright.

### 4.3 HTTP, fetch and Node.js Servers

**How HTTP works** Browsers, apps and test tools talk to servers with HTTP . The client sends a request : a method, a path, headers and sometimes a body. The server sends back a response : a status code, headers and usually a body. Both are plain text you can read.

**fetch and URLs** fetch(url, options) sends a request and returns a promise for the response. It is built into browsers and Node.js. Read the body once with res.json() , res.text() or res.blob() .

**Authentication: 401 against 403** Most APIs need you to prove who you are. A common pattern: log in once, receive a token , then send it on every request in the Authorization: Bearer … header. Browsers usually keep the session in a cookie instead.

**Writing a server with Express** On the server side, Node.js answers requests. Express is the most used library for it: you declare routes (a method plus a path) and a handler that receives the request and builds the response.

**CORS and the same origin policy** A browser keeps websites apart. A page from https://shop.example may freely send requests to other sites, but by default its JavaScript may not read the answers from a different origin (scheme, host and port together).

**Real time: Server-Sent Events and WebSockets** Normal HTTP is "ask, then get one answer". Live order tracking, chat, stock prices and multiplayer games need the server to push updates the moment they happen. Asking again every few seconds (polling) works but wastes requests and is always a little late.

**Node.js core modules** Node.js comes with modules for files, paths, processes and more. Test suites use them to read test data, write reports and run other tools.

**Real world scenario: a mock API for UI tests** The /__test routes exist only in the mock. A test resets the state, sets up exactly the products it needs and can inject a 503 to check that the page shows a friendly error and a retry button. A delay shows whether loading spinners appear.

TRAP `fetch` only rejects when there is no response at all: a refused connection, a DNS error or an abort. A 404 or 500 still fulfils, so always check `res.ok` or `res.status`.

## STEP 5 JavaScript in depth

### 5.1 How JavaScript really works

**Execution contexts and the call stack** Every time a function is called, the engine creates an execution context : a record of its variables, its scope chain and its this . Contexts sit on the call stack . The one on top is running, and when it returns it is popped off.

**Creation phase, hoisting and the scope chain** Before a function runs, the engine scans it and sets up its bindings. var names start as undefined , function declarations are ready to call, and let , const and class exist but sit in the temporal dead zone until their line runs.

**this in depth: five rules and a precedence** The value of this is decided by how a function is called , not where it is written. Five rules cover every case, and when several apply, the one higher in this list wins.

**The Node.js event loop, phase by phase** Chapter 11 showed the browser view: microtasks first, then one task. Node.js splits the task side into phases that run in a fixed order on every turn: timers (setTimeout and setInterval), poll (finished I/O such as files and sockets) and check (setImmediate).

**Inside V8: JIT, hidden classes and inline caches** V8, the engine in Chrome and Node.js, first turns your code into bytecode and runs it with the Ignition interpreter. Functions that run often are compiled to machine code by the optimising compilers ( Sparkplug , Maglev and TurboFan ).

**Garbage collection** JavaScript frees memory for you. V8 uses a generational collector because most objects die young. New objects go into the small young generation , which is cleaned often with a fast copying "scavenge".

**Real world scenario: a crash only in the retry helper** The helper calls action() as a plain function, so this is undefined inside submit . Wrapping it in an arrow or binding it keeps the page object as this . The team also added a lint rule, @typescript-eslint/unbound-method , which flags this mistake in review.

### 5.2 Errors, debugging and logging

**The error types and building your own** Every error has a name , a message and a stack . The built in types tell you what kind of mistake happened, which narrows the search before you read a single line of code.

**Errors in async code** A try block only sees errors thrown while it is on the call stack. A callback that runs later has left it already. With await , a rejected promise turns into an ordinary throw at that line, which is the main reason async and await made error handling simple.

**Throw or return: errors as values** Throwing is right for things that should never happen. For failures you expect, such as a user typing a bad coupon, returning a result object keeps the flow simple and forces the caller to look at both cases.

**Debugging tools** console.log is fine for a quick look, but it hides deep objects and its output is easy to misread. Node.js gives you precise tools for inspecting values.

**Structured logging** Plain text logs are hard to search across many servers or parallel tests. Structured logs write one JSON object per line with a level, a message and fields.

**Source maps** TypeScript, bundlers and minifiers change your code before it runs, so stack traces point at generated files. A source map is a file that maps each generated position back to the original line. Browsers use them automatically in DevTools.

**Real world scenario: triage a red pipeline** Error names and messages carry enough signal to sort most failures automatically. Connection errors and 5xx responses point at the environment, timeouts at timing or locators, assertion errors at the product, and TypeErrors in test code at the tests …

### 5.3 Regular expressions in depth

**How the regex engine matches** JavaScript uses a backtracking engine. It tries the pattern at the first position of the text and, when a step fails, goes back to the last choice it made and tries another. If nothing works, it moves one character on and starts again.

**Lookarounds and backreferences** A lookaround checks what comes before or after a position without including it in the match. (?=x) and (?!x) look ahead, (?<=x) and (?<!x) look behind. A backreference such as \1 or \k<name> matches the same text a group already matched.

**Flags in depth and the lastIndex trap** Flags change how the whole pattern works. The g and y flags also make a regex stateful : it remembers lastIndex , the position after the last match.

**Unicode and every language** [a-z] only knows English. With the u or v flag, \p{L} matches a letter in any script, \p{Script=Devanagari} one script, and \p{Emoji} emoji. The u flag also treats an emoji as one character instead of two halves.

**Replacing, masking and splitting** replace and replaceAll can use $1 , $<name> and $& in the replacement text, or a function that receives every group and returns the new text.

**ReDoS and safe patterns** Some patterns make the engine try an exponential number of ways to split the input. ^(a+)+$ on a long run of "a" that ends with "!" never finishes in practice. An attacker who can send such input to a server can freeze it with one request.

**Validation patterns and testing them** A regex can check the shape of text, not its meaning. A date pattern accepts 31 February, and no practical email pattern follows the full standard. Use a regex for the shape, then real code for the rest, such as Temporal.PlainDate.from for dates.

**Real world scenario: mine the access logs** One pattern with named groups turns each line into an object. Lines that do not match are counted instead of silently dropped, which tells you when the log format changes. The same idea works for test runner output, browser console logs and CI logs.

TRAP Never reuse a regex with the g flag for `test()` across different strings or calls. The second call starts at the old `lastIndex` and can fail on text that clearly matches. Drop the g flag for yes or no checks.

TRAP Danger signs: a quantifier inside a quantified group, such as `(a+)+`, `(\w*)*` or `(a|aa)+`. Safe habits: limit input length first, prefer specific classes, and run the linter rule `regexp/no-super-linear-backtracking` from eslint-plugin-regexp.

### 5.4 Functional programming in JavaScript

**Pure functions and side effects** A pure function returns the same output for the same input and changes nothing outside itself: no global variables, no network, no clock, no random numbers.

**Immutability** Data that never changes cannot be broken by another part of the program. const only stops a variable from being reassigned. Object.freeze stops an object from changing, but only one level deep.

**Currying, partial application and composition** Currying turns f(a, b, c) into f(a)(b)(c) . Fixing the first few arguments, called partial application , creates specialised functions from a general one. Composition builds a function out of smaller ones: pipe runs them left to right, compose right to left.

**Eager and lazy collections** Array methods are eager : each step creates a full new array. Iterator helpers from ES2025 are lazy : they pass one item at a time through all steps and stop as soon as take has enough. On large data that can be thousands of times less work.

**Recursion on real data** JSON responses, the DOM and file systems are trees, and recursion is the natural way to walk a tree. Very deep recursion can overflow the stack, and a trampoline avoids that by returning a function instead of calling it, so a loop does the calls one at a time.

**Real world scenario: a validator built from tiny functions** Each rule is a small pure function that returns a list of problems, and shape and listOf combine them.

### 5.5 Objects in depth and metaprogramming

**Property descriptors, getters and setters** Every property has a descriptor with flags. writable allows changes, enumerable shows it in Object.keys , loops and JSON, and configurable allows deleting it or changing its flags. Accessor properties use get and set functions instead of a value.

**Well known symbols** Built in operations look for special symbol keys on your objects. Symbol.toPrimitive controls conversion to numbers and strings, Symbol.hasInstance controls instanceof , Symbol.toStringTag the type label, and Symbol.iterator (chapter 8) makes an object ...

**Proxy and Reflect** A Proxy sits in front of an object and can intercept almost anything done to it: reading, writing, deleting, in , listing keys, calling and new . Each handler function is a trap .

**Tagged template literals** A tag is a function placed before a template literal. It receives the fixed text pieces and the values separately, so it can escape values, build safe queries or change the text.

**Decorators** A decorator is a function written as @name before a class, method, field or accessor. It receives the original and can replace or wrap it. Test frameworks use decorators for steps, retries, timeouts and dependency injection.

**Real world scenario: a fluent API client** Each property read adds a part to the path, and the verb at the end sends the request. The same trick is used by real clients for large APIs. Pair it with TypeScript types (chapter 24) so the editor suggests only valid paths.

TRAP Proxies add a cost to every operation and can surprise people reading the code. Use them in tools and test frameworks, not in hot paths of application code.

### 5.6 Node.js in depth

**Events and EventEmitter** Much of Node.js is built on EventEmitter : servers, sockets, streams, child processes and test runners. Listeners register with on or once , and emit calls them in order, synchronously.

**Streams and backpressure** A stream moves data in small chunks, so a 5 GB log file never has to fit in memory. Readable streams produce data, Writable streams consume it, and Transform streams change it on the way.

**Buffers, encodings and crypto** A Buffer is a fixed block of bytes, a Uint8Array with extra methods. Text needs an encoding to become bytes: in UTF-8, "é" takes 2 bytes and "₹" takes 3, so byte length and string length differ.

**The built in SQLite database** Node.js ships with node:sqlite , a full SQL database that runs in memory or in a file with no server and no install. It is ideal for seeding test data, checking reports, and learning SQL alongside JavaScript.

**Processes, signals and graceful shutdown** spawn starts another program and streams its output as it arrives, which suits long runs such as a test suite or a dev server. Programs are stopped with signals : SIGINT when you press Ctrl+C and SIGTERM when CI or Docker stops a job.

**Building and testing a command line tool** util.parseArgs reads flags with no extra package, readline reads the file line by line, styleText adds colour that respects NO_COLOR , and the exit code tells CI the result. The bin field in package.json turns the file into a command after npm install .

TRAP An `"error"` event with no listener throws and can crash the process. Every emitter you create, and every stream and socket you use, needs an error listener or a `pipeline` that handles errors for you.

## STEP 6 Testing with JavaScript

### 6.1 Unit Testing: node:test, Jest, Vitest and Mocha

**What a unit test is** A unit test checks one small piece of code, usually one function or one class, on its own. It runs in milliseconds, needs no browser, server or database, and tells you exactly which piece broke.

**Mocks, spies, stubs and fake timers** Real code calls other things: an email service, the clock, the network. A unit test swaps them for test doubles so the test is fast and predictable and never sends a real email.

**Jest** Jest is the most used JavaScript test runner. It has matchers built in ( expect(x).toBe(y) ), table driven tests with test.each , mocks for whole modules, fake timers, snapshots and coverage. By default it uses CommonJS.

**Vitest and Mocha** Vitest has almost the same API as Jest ( vi.fn instead of jest.fn ) but runs ES modules and TypeScript with no set up and is very fast. It is the natural choice for Vite, Vue and modern React projects.

**Running tests in different ways** The same suite is run differently depending on the moment: everything before a merge, one file while you fix it, only smoke tests after a deploy, split across machines in CI. Every runner has options for this.

**Unit testing web (UI) code** UI components can be unit tested without a real browser. jsdom is a browser written in JavaScript that runs inside Node.js: it has document , events and forms, but draws nothing.

**Unit testing API code** API code has two sides to test. Incoming : your routes. supertest sends requests straight into an Express app, without opening a port, and checks the status, headers and body. Outgoing : calls your code makes to other services.

**Real world scenario: turn a bug report into a test** First write a test that reproduces the bug and watch it fail. A failing test proves the test can see the bug.

TRAP Mock what is slow or outside your control, not the code under test itself. A test where everything is mocked only proves that the mocks work.

TRAP A `test.only` committed by mistake silently skips the rest of the suite in some runners. Add a lint rule or run CI with `--forbid-only` (Mocha, Playwright) to block it.

### 6.2 Automated API Testing with JavaScript

**What an API test checks** An API test sends real HTTP requests to a running service and checks the answers. It sits in the middle of the test pyramid: much faster and steadier than a browser test, and it checks the real routes, validation, database and auth together, which unit tests ...

**Testing a running server: a CRUD flow** The most direct API test starts the server, sends requests with fetch and checks each answer. Listening on port 0 lets the system pick a free port, so many suites can run side by side.

**Checking the shape with JSON Schema** Checking every field by hand gets long, and it misses fields you did not think of. A JSON Schema describes the allowed shape once: required fields, types, ranges and whether extra fields are allowed.

**Negative, boundary and data driven tests** Most API bugs hide in input the developer did not expect. Negative tests send wrong data on purpose and check that it is refused with a clear 400, not saved and not answered with a 500. A table of cases with it.each keeps dozens of them readable.

**Authentication, state and conflicts** Real APIs need a login and change data as they go. Log in once in beforeAll , keep the token, and send it with each request.

**API testing with Playwright** Playwright is known for browsers, but its request fixture is also a complete API client. It shares the base URL and config with UI tests, and its webServer option starts the app before the tests and stops it afterwards.

**Response times** A correct answer that takes five seconds is still a bug. Measure each request and look at percentiles , not the average: p95 is the time under which 95% of requests finish. One very slow request barely moves the average but is exactly what a user notices.

**Real world scenario: buying the last two mugs** Subtests with t.test name each step, so a failure says exactly where the flow broke. The last step deletes the product the test created, so the shared environment stays clean for everyone else.

TRAP Tests that depend on each other stop at the first failure and cannot run alone. Use this style for one flow on purpose. For everything else, each test creates its own data.

TRAP Times measured on a laptop or a shared CI machine vary a lot. Use generous limits in functional suites to catch big slowdowns, and leave real load testing to tools such as k6 against a production like environment.

### 6.3 UI Automation: Playwright, Selenium, Cypress and WebdriverIO

**The tools and how they drive a browser** A UI automation tool starts a real browser and acts like a user: it opens pages, types, clicks and reads what is shown. The four big JavaScript tools differ in how they talk to the browser, and that explains most of their strengths and limits.

**Playwright: locators and web first assertions** A Playwright test receives a page , finds elements with locators and checks them with web first assertions . A locator is a recipe, not an element: it is looked up again on every use, so it never goes stale.

**Controlling the network** Playwright sees every request the page makes. page.route can answer a request with fake data or an error, so you can test states that are hard to produce for real: a broken API, an empty list, a very long name.

**Page objects, fixtures and retries** As a suite grows, repeated steps move into page objects (chapter 9) and fixtures . A Playwright fixture prepares something a test needs, here a logged in shop, and passes it in as a parameter. Tests become short and read like a test case.

**Selenium WebDriver** Selenium WebDriver sends each command to a driver program (chromedriver, geckodriver, safaridriver), which controls the browser. Every call is a network round trip and returns a promise, so every line is awaited.

**Cypress and WebdriverIO** Cypress runs inside the browser, in the same loop as your app. Commands such as cy.get are not promises: Cypress queues them and runs them in order, retrying each query and assertion until it passes.

**Real world scenario: a UI suite in CI** Sharding splits the tests across four machines. fail-fast: false lets every shard finish, so one failure does not hide others. The report is uploaded even when tests fail, and the traces inside it show exactly what the browser saw.

TRAP Retries keep a pipeline green while you investigate, they are not a fix. With `trace: "on-first-retry"`, Playwright records a trace of the retry: every action, DOM snapshot, network call and console message, which usually shows the cause in minutes.

TRAP Avoid `driver.manage().setTimeouts({ implicit })` together with explicit waits. Mixing the two makes waits add up in surprising ways. Pick explicit waits.

## STEP 7 TypeScript

### 7.1 TypeScript fundamentals

**What TypeScript is and how it runs** TypeScript is JavaScript with type annotations. The compiler tsc reads the types and reports mistakes before the code runs. The types are then erased : at run time it is plain JavaScript.

**Everyday types** Most of the time TypeScript infers the type from the value, so you annotate function parameters, public APIs and empty containers, not every variable. Use readonly for data that must not change, ? for optional properties, and tuples for fixed length lists.

**Functions** Annotate parameters, and let the return type be inferred or write it for public functions. Optional parameters use ? , defaults give both a type and a value, and rest parameters are typed as arrays.

**interface and type** Both interface and type name an object shape. Interfaces can be extended with extends and are merged when declared twice, which suits public object shapes. Type aliases can name anything: unions, tuples, functions and combinations with & .

**Unions and narrowing** A union such as string | number means "one of these". Before you use a union, you narrow it with a check that TypeScript understands: typeof , instanceof , in , equality, or a shared literal field. Inside each branch, TypeScript knows the exact type.

**Enums and their modern alternative** An enum creates a real object at run time, so it is not erasable and fails with erasableSyntaxOnly and with Node.js type stripping. The modern alternative is a plain object marked as const plus a union type made from its values.

**Strict options** "strict": true turns on a family of checks. The two that catch the most bugs are strictNullChecks , which makes you handle undefined and null , and noImplicitAny , which stops untyped parameters.

**Real world scenario: typed results from untyped JSON** JSON.parse really returns unknown data, whatever its declared type. A type predicate such as x is Result checks it at run time and tells TypeScript the result, so everything after the filter is fully typed and the bad rows are counted instead of breaking the ...

TRAP A project without `strict` lets most of these bugs through, so TypeScript gives a false sense of safety. Turn it on from day one. When moving an old JavaScript project, turn on `checkJs` and fix files one folder at a time (chapter 25).

### 7.2 Advanced TypeScript types

**Generics** A generic is a type parameter, written <T> , that is filled in when the function or type is used. It keeps the connection between inputs and outputs: first of a number list is a number. A constraint with extends says what T must at least have.

**typeof, keyof and indexed access** typeof value turns a value into a type, keyof T gives the union of its keys, and T[K] reads the type of one property. Together they let a function accept only real keys and return the right type for each one.

**Utility types** TypeScript ships with type helpers that change existing types instead of writing new ones by hand.

**Mapped and conditional types** A mapped type loops over keys: { [K in keyof T]: ... } . It can rename keys with as and add or remove readonly and ? .

**Template literal types** Template literal types describe patterns of strings, built from other string types. They type routes, CSS values, event names and test ids, and the compiler checks every literal you write.

**satisfies** An annotation such as const x: Record<string, Project> replaces the type of the value with the wider annotation, so the exact keys are lost. satisfies checks the value against the type but keeps the precise type that was inferred.

**Exhaustive checks with never** With a discriminated union, a switch on the shared field narrows each case. Passing the leftover value to a function that takes never makes the check exhaustive : add a new kind to the union and every switch that forgot it becomes a compile error.

**Assertion functions and branded types** An assertion function with asserts value is T narrows the type after the call, or throws. Test helpers such as expect(x).toBeDefined() cannot narrow types, but your own assertion functions can.

**Real world scenario: typed API routes** A conditional type with infer pulls :id out of the route string, and the Routes map gives the response type. A missing parameter or a typo such as stok is now a compile error.

TRAP Look at the `leaked` line: spreading an object with extra fields into a typed variable is allowed, so a password can slip into a "public" object. Types describe what must be there, not what must be absent. API tests must still check that secret fields are not in the response.

### 7.3 TypeScript classes and real projects

**Classes in TypeScript** TypeScript adds access modifiers to classes. public is the default, protected allows subclasses, and private allows only the class itself.

**Decorators** TypeScript 5 supports the standard decorators proposal. A method decorator is a function that receives the original method and a context object, and returns a replacement. Decorators stack: the one closest to the method wraps first.

**Declaration files and augmentation** A declaration file ( .d.ts ) contains only types. It describes JavaScript code that has no types of its own. Most npm packages ship one, and others have one in @types/... .

**Checking JavaScript with JSDoc** You can get most of the safety of TypeScript without renaming a single file. With checkJs , or // @ts-check at the top of a file, TypeScript reads types from JSDoc comments and reports the same errors.

**Moving a project to TypeScript, and the tools around it**

**Real world scenario: a typed page object hierarchy** An abstract path forces every page to declare its URL, protected el() keeps raw locators inside page objects, and returning DashboardPage from loginAs makes the next step in a test auto complete. In a real project, PageLike is Playwright's Page type.

TRAP Path aliases such as `@pages/login` in `paths` are only understood by tsc and by tools that read tsconfig. Plain Node.js does not resolve them. Use `package.json` `"imports"` with `#pages/*` instead, which works everywhere.

## 7.4 Test automation in TypeScript

**Why write tests in TypeScript** Test code is code. It has typos, wrong arguments and missing await s, and in JavaScript you only find them when the test runs, often in CI, often after a slow browser start. TypeScript finds them while you type.

**Unit tests in TypeScript** Node.js 24 and later remove the types and run .ts files directly, so node --test needs no build step. Vitest reads TypeScript too, without any set up. In both cases the runner only strips the types and does not check them, so run tsc as its own step.

**Type tests** Shared helpers, builders and API clients are used by hundreds of tests, so their types are part of their contract. expectTypeOf from Vitest (and the expect-type package it is built on) asserts on types instead of values. Nothing runs.

**Playwright in TypeScript** Playwright runs TypeScript without any set up, and most Playwright documentation is written in it. Two things become much better with types. Page objects can limit their inputs, for example to real product names.

**A typed API client with run time checks** TypeScript types disappear at run time, so they cannot check JSON from a server. A schema library such as zod does both jobs at once.

**TypeScript in CI** Put the type check in front of everything slow. It needs no browser and no server, and it fails in seconds.

**Real world scenario: typed test data builders** Each builder holds the defaults in one place and gives every object a fresh id, so tests stay independent. Overrides are Partial<T> , so a test only writes the fields it cares about, and a misspelt field or a wrong role does not compile.

TRAP Removing types is not checking types. A test file full of type errors still runs green under `node --test`, Vitest or `@swc/jest`. Only `tsc` (or ts-jest) reports them. That is why every example on this page runs `tsc` first.

## 7.5 How the TypeScript type system decides

**Structural typing: shapes, not names** In Java or C#, a class is a Reporter only if it says implements Reporter . TypeScript is structural : any value with the right members fits, whatever it is called. This matches how JavaScript code is really written, with plain objects passed around.

**Literal types and widening** A value can have a very exact type. const m = "GET" has the type "GET" , a literal type . A let may change later, so TypeScript widens it to string . Object properties are widened too. as const stops widening and makes everything readonly.

**any, unknown and never** any turns checking off for a value and everything you reach through it, so mistakes spread silently. unknown accepts any value too, but you must check it before use.

**Function compatibility and variance** When is one function type usable as another? Fewer parameters is fine, because JavaScript ignores extra arguments. A callback that returns a value is fine where void is expected. Parameter types are where it gets subtle.

**Index signatures and the strictest options** An index signature , { [key: string]: number } or Record<string, number> , describes objects used as dictionaries. By default TypeScript pretends every key exists. Two options, not included in strict , make it honest.

**Tuples: fixed length arrays with names** A tuple is an array with a fixed length and a type per position. Elements can have labels, be optional, or be a rest of many.

**Overloads and this** Overloads give one function several typed call signatures, so the return type follows the argument type.

**Recursive types** A type may refer to itself. That is how trees are described: JSON, file systems, menus, nested config. Recursive conditional types go further and compute types such as every dot path in an object.

## 7.6 TypeScript project setup: tsconfig, modules and builds

**tsconfig.json in depth** Every TypeScript project is driven by tsconfig.json . It answers four questions: which files to check ( include , exclude , files ), how strict to be, what JavaScript to produce, and how imports are found.

**ES modules and module resolution** The module and moduleResolution options tell tsc which rules the real runtime will use to find files. With nodenext it copies Node exactly: ES module imports need the file extension, and packages are found through their exports field.

**Subpath imports instead of ../../..** Deep test folders end up with imports like ../../../lib/price . Node's subpath imports fix that with an imports field in package.json, and both Node and tsc understand it without extra tools.

**Erasable syntax and running .ts files directly** Node.js 23.6 and later run .ts files directly by replacing types with spaces, without checking them. That only works for TypeScript features that leave no JavaScript behind.

**Declaration merging and namespaces** TypeScript merges some declarations that share a name. Two interface blocks become one, which is how libraries let you add fields to their types. A function and a namespace can merge too, giving a function with extra properties.

**isolatedDeclarations and fast declaration files** A library publishes .d.ts files that describe its exports. Normally only tsc can write them, because it must infer return types.

**Project references for big repositories** In a large repository with many packages, checking everything every time is slow. Project references split the code into projects that build separately and in order. tsc -b (build mode) checks only what changed since the last build.

## 7.7 What is new in TypeScript: 5.0 to 7.0

**Three years of TypeScript in one page** TypeScript ships a new version every few months, and each brings small features that save real time. This chapter goes release by release from 5.0 (2023) to 7.0 (2026). Every example is checked by the TypeScript 7 compiler and run on Node.js 26.

**TypeScript 5.0: const type parameters** Before 5.0, a generic function that should keep exact strings needed callers to write as const . A const type parameter does it for every caller. 5.0 also introduced standard decorators, covered in the projects chapter.

**TypeScript 5.2 and 5.3: using and import attributes** using is explicit resource management from a JavaScript proposal that TypeScript supports since 5.2 and Node.js runs natively. When the block ends, normally or by an error, the object's [Symbol.dispose]() runs.

**TypeScript 5.4 and 5.5: smarter inference** NoInfer<T> marks an argument that must match T but must not help decide what T is. Without it, a wrong default such as "safari" would simply widen T to include it. 5.4 also keeps narrowing inside callbacks created after the last change to a variable.

**TypeScript 5.6 to 5.9: fewer silent mistakes** 5.6 reports conditions that can never be false, such as a function used without calling it. This is a classic bug in test helpers: if (isLoggedIn) is always true. 5.6 also typed the iterator helper methods such as filter , map and toArray on iterators.

**TypeScript 6.0: the bridge release** TypeScript 6.0 (March 2026) is the last version of the compiler written in JavaScript.

**TypeScript 7.0: the native compiler** TypeScript 7.0 is the compiler ported to Go. The language is the same, but builds are typically 8 to 12 times faster, and the VS Code code base went from 125.7 seconds to 10.6 seconds. Checking can run on several CPU cores.

## STEP 8 Expert and latest

### 8.1 Modern JavaScript: ES2020 to ES2027

**How JavaScript grows every year** Every June, Ecma publishes a new edition of the standard: ES2020, ES2021 and so on. A feature joins only after it reaches stage 4 at TC39, which needs two shipping browser engines and a full test suite.

**ES2020 to ES2023: safer access and copies** Optional chaining and nullish coalescing removed most of the "cannot read properties of undefined" crashes from everyday code. The ES2023 change by copy methods return a new array and leave the original alone, which makes test data safe to reuse.

**ES2024 and ES2025: grouping, iterators, safer regexes** Grouping a list, a promise you resolve from outside, lazy iterator helpers, real set operations and a safe way to put user text into a regular expression all arrived in the last two editions.

**ES2026 and ES2027: using, Error.isError, upsert and more** The June 2026 edition, ES2026, is a collection of small tools that remove common helper functions from real projects:

**Temporal: dates done right** The old Date object mixes up time zones, months that start at 0 and objects that change in place.

**What comes next, and how to check before you use it** A feature you read about may not exist where your code runs. Old phones, a company laptop with an old browser, or a CI image with an older Node.js all lag behind. Professionals do not guess from version numbers.

**Real world scenario: modernise safely** The rewrite is a third of the size. Running both versions on the same inputs, including an empty list, and comparing the results is a simple form of characterisation testing : it proves the behaviour stayed the same before you delete the old code.

TRAP `new RegExp(userText)` breaks or misbehaves when the text contains `( ) . ? $`. `RegExp.escape` fixes that, which matters for tests that search a page for product names or prices.

### 8.2 Expert JavaScript: performance, security and test quality

**Performance: count first, then measure** Most slow JavaScript is slow because of the algorithm, not the engine. A loop inside a loop over the same list does about $n^2 \div 2$ steps, while a Set lookup does about n . Count steps to compare ideas, and measure time only to confirm.

**Memory: leaks and weak references** JavaScript frees memory that nothing can reach any more. A leak happens when something you forgot still points at old data: a listener that was never removed, a cache that only grows, or a timer that keeps running.

**Patterns experts reach for** A pattern is a known shape for a common problem. These four appear in application code and in test frameworks alike.

**Workers: real parallel work** JavaScript runs your code on one thread, so a long calculation blocks everything else. Worker threads in Node.js and Web Workers in the browser run code on separate threads and talk to the main thread with messages.

**Security every JavaScript developer should test** Cross site scripting (XSS) happens when user text is turned into HTML. The safe defaults are textContent , framework templates that escape by default, and a Content Security Policy header that blocks inline scripts.

**Test quality: properties, mutants and flaky tests** Example based tests check the inputs you thought of. Property based testing generates hundreds of inputs and checks a rule that must always hold. Libraries such as fast-check also shrink a failure to the smallest input. The seed makes every failure repeatable.

**Real world scenario: a faster CI pipeline** Splitting by position put the slow checkout and payments files on the same machine. Sorting by duration and filling the lightest shard first cuts the wait by more than three minutes with the same three machines.